IceBridge: an Authenticated Peer-to-Peer Service Mesh over UDP, with Rendezvous on the BitTorrent DHT

Technical White Paper and Implementer's Specification

Angel Leon — FrostWire

November 2026

Abstract

IceBridge is the peer-to-peer network that FrostWire for desktop 7.1 and FrostWire for Android 3.2 use for distributed torrent search, catalog browsing and torrent metadata transfer. It has two parts. The BitTorrent Mainline DHT is the directory: nodes publish signed identity records and index manifests as BEP 44 mutable items, and announce themselves on well-known BEP 5 topics. IceBridge proper is a transport and service mesh: a reliable UDP protocol ("rUDP") whose sessions are bound to Ed25519 identities, a bounded multi-hop relay, and an application envelope that carries any protocol identified by a 16-bit number. Distributed search is one such protocol. A node answers searches only from the torrents it is actively sharing, and every answer is signed by the node that holds the torrent. This paper describes the problem IceBridge solves, the design, and the wire formats, constants and procedures of the Java implementation shared by FrostWire 7.1.3 for desktop and FrostWire 3.2.3 for Android (IceBridge software version 1.1.0, rUDP protocol version 2). It is written so that an independent implementation, such as the planned Rust one, can interoperate. Where the code and earlier design documents disagree, this paper follows the code and records the difference. One difference matters more than the others: rUDP version 2 authenticates every packet but does not encrypt anything. Payloads, including search keywords and results, travel in clear text.

Contents

1 Problem **4**

2 Design goals and non-goals **4**

2.1 Goals 4

2.2 Non-goals 5

3 Architecture overview **5**

3.1 Planes 5

3.2 Deployments 6

3.3 Roles and capabilities 6

4 Identity **6**

4.1 Keys 6

4.2 Identity record 7

5 Transport: rUDP version 2 **8**

5.1 Packet header 8

5.2 Handshake 8

5.3 Packet protection . . . 10
5.4 Encryption status . . . 11
5.5 Reliability . . . 11
5.6 Fragmentation . . . 12
5.7 Path migration . . . 12
5.8 Admission control . . . 12
5.9 NAT traversal . . . 12
5.10 Relay frame . . . 13

6 Discovery and the DHT 13
6.1 Why the DHT and IceBridge fit together . . . 13
6.2 BEP 44 mutable items . . . 14
6.3 BEP 5 topics . . . 15
6.4 Discovery pass . . . 15
6.5 Host cache . . . 16
6.6 LAN multicast beacon . . . 16
6.7 DHT index manifest . . . 16

7 Relay mesh, topology and control API 16
7.1 The IBP1 application envelope . . . 16
7.2 Topology . . . 16
7.3 Control API . . . 18

8 Services 18
8.1 Distributed search (protocol 1) . . . 19
8.2 Catalog browsing (protocol 11) . . . 20
8.3 Torrent metadata fetch (protocol 3, `TORRENT_FETCH`) . . . 21

9 Seeding is publishing: the consent model 21

10 Karma and reputation 22
10.1 Entries . . . 22
10.2 Bitcoin anchoring . . . 22
10.3 Publication . . . 23
10.4 Use in ranking . . . 23
10.5 Limitations in the current code . . . 23

11 Security and threat model 24
11.1 Assets and adversaries . . . 24
11.2 What the design resists . . . 24
11.3 What it does not resist . . . 24

12 Implementation notes for new implementations 25
12.1 Encodings . . . 25
12.2 Protocol behaviour . . . 25

12.3 Compatibility checklist 26

13 Status and future work **26**

A Wire formats and constants **28**

A.1 Strings and identifiers 28

A.2 Limits and timers 28

A.3 Test vectors 29

A.4 Discrepancies between documents and code 30

1 Problem

BitTorrent moves content without a central server. Finding content does not work that way. A user who wants a torrent usually searches a web site that keeps a central index of `.torrent` files or magnet links. These index sites are single points of failure:

- They go offline. Many large index sites have closed after legal action, domain seizures, or because the operator stopped paying for hosting. Their indexes vanish with them.
- They are easy to block. An ISP or a national firewall can block a handful of domain names and cut off search for millions of users, while the DHT and the swarms keep working.
- They go stale. A central index lists torrents that nobody seeds any more. Users download dead magnets and wait.
- They are a trust bottleneck. Users have to trust the site's moderation, uploader badges and comments, and the site sees every query.

The Mainline DHT (BEP 5) already gives every BitTorrent client a decentralized way to find peers for a known info-hash. It cannot answer "which torrents match these keywords". The DHT stores small values keyed by hashes. It has no keyword search, and storing an inverted index in it would overload the nodes that happen to be close to popular keywords.

IceBridge closes that gap. It builds on what BitTorrent clients already run: the DHT, the peers' own libraries of shared torrents, and their identities. The DHT is the directory and IceBridge is the transport. Keyword queries go directly to the peers that hold the content, and answers come back signed by those peers.

The maintainer's long-term goal is a network that, once enough peers join, makes central torrent index sites largely unnecessary. Search results are fresh by construction, because a peer only answers with torrents that its BitTorrent session is actively sharing. In FrostWire's words: *seeding is publishing*.

2 Design goals and non-goals

2.1 Goals

1. **No required central server.** Any node can find others through the DHT and the LAN. FrostWire's own servers make discovery faster but are optional (Section 6).
2. **Censorship resistance through decentralization.** No single domain or IP address needs to be reachable. Node endpoints are published on the DHT under topic hashes, and identities are self-certifying Ed25519 keys.
3. **Authenticated communication.** Every rUDP packet after the handshake is signed by its sender and bound to both endpoints and to the handshake transcript. Application messages such as search requests, search results, catalog pages and torrent metadata chunks carry their own end-to-end Ed25519 signatures. A relay cannot forge or alter them without detection.
4. **Protocol-agnostic transport.** IceBridge carries opaque payloads tagged with a 16-bit protocol identifier (Section 7.1). Search is protocol 1. Catalog browsing, metadata fetch, keyword digests and node metadata are other protocols. Chat, pub/sub, AI and file sync have reserved identifiers.
5. **Bounded resource use.** Every queue, cache, rate limiter, fan-out, hop count, fragment group and payload has a hard cap. Appendix A lists them.
6. **Fresh results.** Only torrents active in the node's BitTorrent session are served (Section 9).

7. **Consent for bulk listing.** Answering keyword searches is part of joining the network. Listing a node's whole catalog to browsers and crawlers is a separate setting.

2.2 Non-goals

1. **Content transfer.** IceBridge never carries torrent payload data. Downloads use ordinary BitTorrent. IceBridge carries only small control messages, search results, and `.torrent` metadata of at most 256 KiB.
2. **Anonymity.** A node's rUDP endpoint is public on the DHT. Peers it talks to see its IP address. IceBridge is not Tor or I2P.
3. **Confidentiality (in version 2).** The current protocol does not encrypt. Identities carry an X25519 public key and the code reserves key-derivation labels, but no encryption is implemented (Sections 5.4 and 13).
4. **A global search index.** No node holds a complete index. Search reaches the peers that a bounded fan-out and keyword digests select.

3 Architecture overview

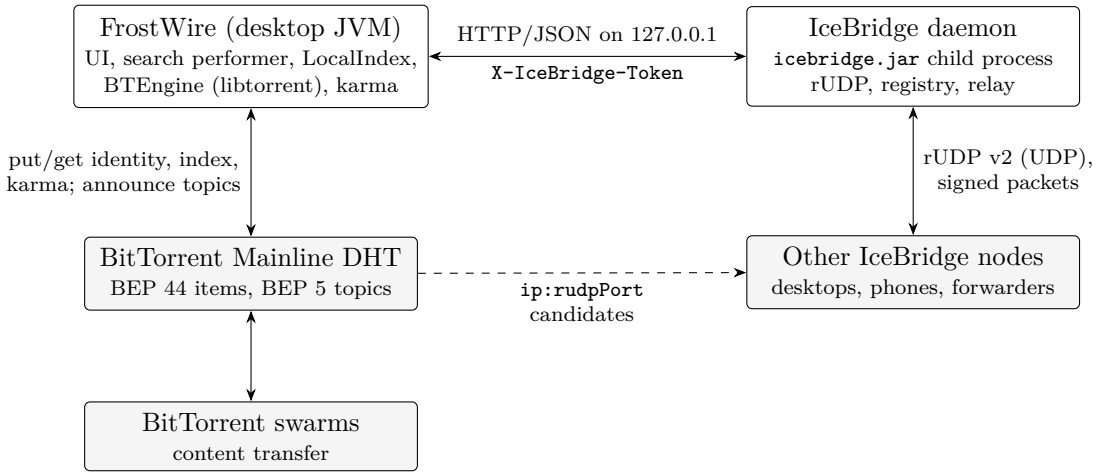


Figure 1: Processes and planes on a FrostWire desktop. On Android (3.2.x) the IceBridge server runs inside the app process, and the app uses the same loopback control API. A standalone forwarder runs the daemon alone, with its own DHT session.

3.1 Planes

IceBridge has four planes.

Directory plane (DHT). Signed identity records, catalog index manifests and karma chains are BEP 44 mutable items keyed by the node's Ed25519 public key. Endpoints are announced on four BEP 5 topics. See Section 6.

Transport plane (rUDP). One UDP socket per node, default port 6889 for servers and an automatically chosen port for desktops. Authenticated sessions, reliability, fragmentation, path migration and relaying. See Section 5.

Control plane. A loopback HTTP/1.1 API through which the application registers identities, sends payloads, polls received payloads and asks the daemon to probe endpoints. See Section 7.3.

Application plane. Protocols carried in the IBP1 envelope: search, catalog, metadata, digests and node metadata. See Sections 7 and 8.

3.2 Deployments

- **Desktop.** FrostWire launches `icebridge.jar` as a child JVM with `--no-dht`. The parent process owns the DHT through its BitTorrent session, owns the local index, and answers searches. The child owns the UDP socket and the mesh.
- **Android.** FrostWire for Android implements IceBridge since version 3.2.0 (September 2026); the current release is 3.2.3. Phones and tablets are full IceBridge nodes that speak the same wire protocol as desktops and forwarders, so they search, answer searches, relay, and browse catalogs across the same mesh. The IceBridge server runs inside the app process rather than as a child JVM, with an automatically chosen rUDP port, a registry of up to 500 peers and a 180 s peer TTL. The app still talks to it through the same control API on an automatically chosen loopback port. The node's role defaults to `BOTH`; users can choose `CLIENT`, and the role profile of Table 9 is applied. The app's own BitTorrent session publishes the identity record and index manifest and announces on the DHT. On Wi-Fi the app holds Android's multicast lock while IceBridge runs, so that it can send and receive LAN beacons. Discovery and announces run every 300 s. A phone can instead use a *remote* IceBridge node (an HTTPS URL and token), in which case it runs no node of its own and announces nothing.
- **Standalone forwarder.** `icebridge.jar` runs alone, for example on a cloud host such as `virginia1.frostwire.com`. It starts a minimal libtorrent session with only the DHT enabled, so that it can announce itself and be found without any FrostWire GUI.

3.3 Roles and capabilities

A node has one of three roles: `FORWARDER` accepts inbound sessions and forwards traffic; `CLIENT` only initiates; `BOTH` does both. In this paper an *ultrapeer* is a node with role `FORWARDER` or `BOTH`. Roles map to a 64-bit capability mask (Table 1) that identity records and `NODE_META` messages advertise.

Bit	Value	Name	Meaning
0	1	`RELAY`	forwards mesh traffic for others
1	2	`SEARCH`	answers distributed search
2	4	`INDEX`	keeps a local shared-torrent index
3	8	`STORE`	reserved (storage)
4	16	`DHT`	participates in the DHT
5	32	`TORRENT`	runs a BitTorrent session
6	64	`AI`	reserved
7	128	`PUBLIC_CATALOG`	answers catalog listing requests

Presets: `FORWARDER` = 17 (0x11), `CLIENT` = 54 (0x36), `BOTH` = 55 (0x37).
`fromRole` upper-cases with `Locale.ROOT`; unknown or null roles map to 55.

Table 1: Capability bits (`NodeCapabilities`).

4 Identity

4.1 Keys

Every node has a long-lived identity:

- An **Ed25519** key pair (RFC 8032). The 32-byte public key *is* the node's identity everywhere in IceBridge: rUDP sessions, routing, DHT item keys, and signatures on search results, catalog pages, metadata and karma. The key pair is derived from a 32-byte seed exactly as RFC 8032 specifies (libtorrent's `ed25519_create_keypair`: SHA-512 of the seed, clamped), so standard libraries such as `ed25519-dalek` produce the same public key from the same seed.
- An **X25519** key pair (RFC 7748). It is generated randomly and independently of the Ed25519 seed, published in the identity record, and *not used* by any protocol today. It is reserved for future encryption.
- A **node ID** of 20 bytes, defined as `SHA-1(ed25519_pub)`. It is published but never checked by receivers, and it is unrelated to the node's libtorrent DHT node ID.

Proof of work. When FrostWire creates a new identity it draws random seeds until `SHA-1(ed25519_pub)` has at least 20 leading zero bits, counted from the most significant bit of byte 0 (`KarmaConstants.IDENTITY_DIFFICULTY = 20`). This costs about 2^{20} key generations. The check is *not* enforced on remote peers, on imported identities, or on identities restored from a seed phrase. Today it is informational, shown in the Identity settings as "identity difficulty".

Seed phrase. The 32-byte Ed25519 seed is shown to the user as a 24-word BIP39 English mnemonic. The seed itself is the BIP39 entropy: 256 bits plus an 8-bit SHA-256 checksum. No PBKDF2 stretching, passphrase or derivation path is applied. Input is normalized with NFKC, lower-cased with `Locale.ROOT`, and split on runs of commas, semicolons and white space. Exactly 24 words are required and the checksum is checked. Restoring from the phrase recreates the Ed25519 key and generates a *new* X25519 key.

Storage. The identity lives in `<settings>/libtorrent/identity.dat`. The file is four records, each a 4-byte big-endian length followed by DER bytes: Ed25519 PKCS#8 private key, Ed25519 X.509 public key, X25519 PKCS#8 private key, X25519 X.509 public key. There is no magic number, version or encryption. The Ed25519 seed is at bytes 16–47 of the PKCS#8 record; the JDK prefix is `30 2e 02 01 00 30 05 06 03 2b 65 70 04 22 04 20`. Importing or restoring an identity keeps one backup, `identity.dat.bak`. There is no signed key rotation or revocation.

Raw key wrapping. Java verifies raw 32-byte Ed25519 keys by prefixing the X.509 SubjectPublicKeyInfo header `30 2a 30 05 06 03 2b 65 70 03 21 00`. Implementations with raw-key APIs do not need this.

4.2 Identity record

A node describes itself in a signed *identity record*. The record is a bencoded dictionary. Its canonical form follows BEP 3 bencoding with these extra rules, which the Java decoder (`util/Bencode`) enforces on every signed structure: keys sorted by their raw bytes, unique; integers in shortest form, no `-0`, no `Long.MIN_VALUE`; no trailing bytes; nesting depth at most 32; strings at most 1 MiB; at most 65,536 items.

Signature. The signature is pure Ed25519 (no pre-hash, no context) by the identity key over the canonical bencoding of the dictionary *without* the `sig` key and containing only the keys defined for the record's version `v`. A verifier must:

1. decode the dictionary and reject non-canonical bencoding;
2. check `1 <= v <= 4`, before narrowing it to a 32-bit integer;

Key	Type	Since	Value
`caps`	int	v3	capability mask, Table 1
`ed25519_pub`	str	v1	standard Base64 of the 32-byte key, no padding (43 chars)
`first_seen`	int	v1	Unix seconds, > 0
`ib_ver`	str	v4	IceBridge software version, `"1.1.0"`
`last_seen`	int	v1	Unix seconds, $\geq$ `first_seen`
`node_id`	str	v1	lowercase hex of the 20-byte node ID (40 chars)
`role`	str	v2	`CLIENT`, `FORWARDER` or `BOTH`
`rudp_port`	int	v2	rUDP port
`sig`	str	v1	standard Base64 of the 64-byte signature, no padding (86 chars)
`utp_port`	int	v1	legacy port; set to the rUDP port by current writers
`v`	int	v1	record version, 1–4; writers emit 4
`x25519_pub`	str	v1	standard Base64 of the X25519 key, no padding

Table 2: Identity record fields. Keys are shown in canonical (sorted) order.

3. read the fields for that version: v1 has no `rudp_port`, `role`, `caps` or `ib_ver`; missing optional fields default to role `BOTH`, port 0, caps from the role, and version `""`;
4. check lengths: node ID 20, keys 32, signature 64; `first_seen > 0`; `last_seen >= first_seen`;
5. *re-encode* the parsed fields in canonical form and verify the signature over those bytes with the embedded `ed25519_pub`;
6. when the record came from the DHT, check that `ed25519_pub` equals the DHT item key.

Because the signature covers re-encoded fields rather than the received bytes, unknown keys are ignored and unsigned. A new field therefore needs a new version number. Upper-case hex, padded Base64 or URL-safe Base64 in a record make verification fail.

The record has no expiry. Receivers do not check the node ID, the proof of work, the role string or the capability mask. The publisher re-signs the record on every publication with `first_seen = last_seen = now`.

5 Transport: rUDP version 2

rUDP is a small reliable-datagram protocol over one UDP socket. It gives each pair of nodes an *association* (session) that is bound to both Ed25519 identities. It provides in-order reliable delivery, fragmentation of payloads up to 256 KiB, NAT rebinding, and a relay frame for multi-hop delivery. Version 2 replaced the unauthenticated version 1. There is no downgrade: version 1 packets are dropped.

5.1 Packet header

All integers are big-endian (network order). The header is 22 bytes.

A receiver silently drops a datagram that is shorter than 22 bytes, has the wrong magic or version, has an unknown type, declares a payload longer than 1152 bytes (`MAX_WIRE_PAYLOAD` = 1024 + 128), or whose length does not exactly match the remaining bytes.

5.2 Handshake

The initiator picks a random CID that is not 0, not -1, and not already in use. Both sides use the same CID for the association. Handshake packets have sequence 0 and ack-through 0.

Offset	Size	Field	Value
0	2	magic	`0x46 0x77` (“Fw”)
2	1	version	2
3	1	type	Table 4
4	8	connection ID (CID)	signed 64-bit, chosen by the initiator
12	4	sequence	unsigned 32-bit semantics
16	4	ack-through	highest in-order sequence received from the peer
20	2	payload length	unsigned 16-bit; at most 1152
22	n	payload	exactly payload-length bytes

Table 3: rUDP packet header (`RudpPacket`, `RudpPacketCodec`).

Code	Type	Signed by	Use
0x01	`HELLO`	body sig	initiator opens a handshake
0x02	`HELLO_ACK`	body sig	responder answers
0x03	`DATA`	packet	one application payload of at most 1024 bytes
0x04	`DATA_ACK`	packet	cumulative acknowledgement; empty body, sequence 0
0x05	`HOLE_PUNCH`	—	**disabled in v2**: dropped on receipt, never sent
0x06	`HOLE_PUNCH_RESPONSE`	—	**disabled in v2**: dropped on receipt, never sent
0x07	`RELAY`	packet	relay frame (Section 5.10)
0x08	`RELAY_RESPONSE`	packet	last hop of a relayed delivery
0x09	`DATA_FRAG`	packet	fragment of a larger payload
0x0A	`DATA_END`	packet	last fragment
0x0B	`HELLO_FINISH`	packet	initiator confirms the handshake
0x0C	`HELLO_READY`	packet	responder confirms the handshake
0x0D	`PATH_CHALLENGE`	packet	32 random bytes to a new address
0x0E	`PATH_RESPONSE`	packet	echo of the challenge

Table 4: rUDP packet types. “Packet” means the packet is protected as described in Section 5.3.

Handshake signature input. `HELLO` and `HELLO_ACK` carry a body followed by a 64-byte Ed25519 signature over:

```
"FrostWire-IceBridge-rUDP-v2"   27 bytes, US-ASCII
u8   version (2)
u8   packet type (0x01 or 0x02)
i64  CID
body
```

`HELLO` payload (168 bytes)			`HELLO_ACK` payload (192 bytes)		
Off	Len	Field	Off	Len	Field
0	32	initiator Ed25519 pub	0	32	responder Ed25519 pub
32	32	intended responder pub, or zeros	32	32	initiator pub (HELLO[0..32])
64	32	random challenge n_A	64	32	SHA-256 of the full HELLO payload
96	8	Unix time, seconds (i64)	96	32	random challenge n_B
104	64	signature	128	64	signature

Table 5: Handshake payloads (`RudpAuth`).

Responder rules. The responder accepts a `HELLO` only if: the payload is exactly 168 bytes; the timestamp is within ±60 seconds of its clock; the signature verifies under the embedded initiator key; the intended-responder field is all zeros or equals its own key; and the initiator is not itself. A zero intended-responder field is *identity discovery*: the initiator only knows an `ip:port` and learns the responder’s key from the answer. This proves that the endpoint holds the key. It does not

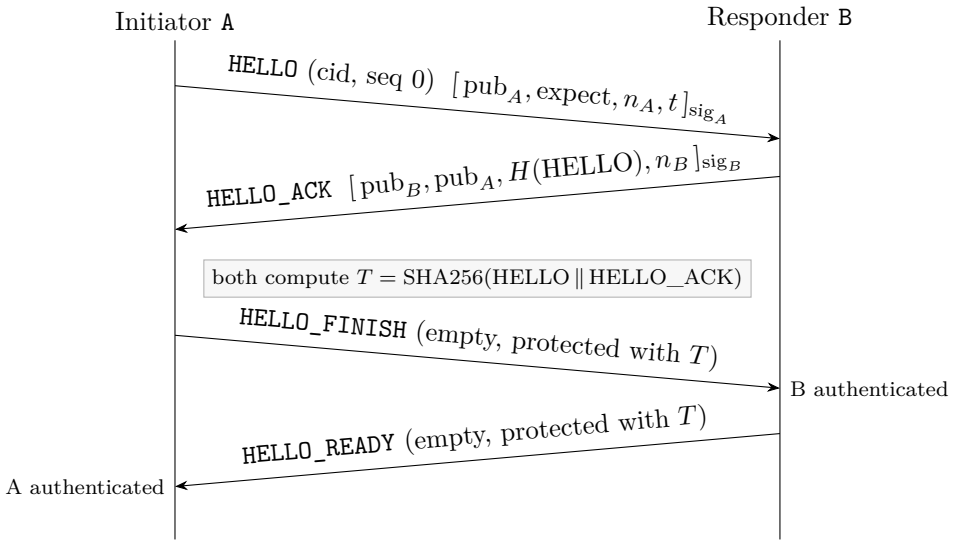


Figure 2: rUDP v2 handshake. "Protected" means signed as in Section 5.3.

prove that the endpoint is a particular trusted node. If the same initiator retransmits an identical `HELLO` from the same address, the responder resends its stored `HELLO_ACK`.

Initiator rules. The initiator accepts a `HELLO_ACK` only if it is exactly 192 bytes, bytes 32–63 equal its own key, bytes 64–95 equal SHA-256 of the `HELLO` it sent, the responder key satisfies the intended-responder field, the signature verifies, and the packet comes from the address it dialled.

Transcript and confirmation. Both sides compute the transcript hash

$$T = \text{SHA256}(\text{HELLO payload} \,\|\, \text{HELLO_ACK payload})$$

over the complete payloads, signatures included. The initiator sends an empty protected `HELLO_FINISH`. The responder marks the association authenticated and answers with an empty protected `HELLO_READY`. The initiator then marks it authenticated. Data packets (sequence ≥ 1) are only sent after authentication. Both nodes record the peer's observed address in their peer registry.

Simultaneous open. If both nodes dial each other at once, each has an outbound initiator session when the other's `HELLO` arrives. The node with the numerically *lower* public key (compared as unsigned bytes) ignores the inbound `HELLO` and stays the initiator. The other node drops its own attempt, becomes the responder, and moves its queued data into the new association. A fresh, valid `HELLO` from a peer that already has an authenticated association, or whose earlier handshake stalled, replaces the old association. This is how a restarted peer reconnects.

5.3 Packet protection

Every packet other than `HELLO` and `HELLO_ACK` carries its body followed by a 64-byte Ed25519 signature by the sender's identity key over:

```
"FrostWire-IceBridge-rUDP-v2"   27 bytes
u8   version (2)
u8   packet type
32   sender Ed25519 pub
32   recipient Ed25519 pub
32   transcript T
i64  CID
```

```
i32  sequence
i32  ack-through
i32  body length
body
```

The header's payload length is the body length plus 64. A receiver looks up the association by CID. It rejects the packet if there is no association or transcript, if ack-through is greater than the highest sequence it has sent, if the global packet rate limit is exhausted, or if the signature does not verify. All of these checks happen before any session state or routing changes. Addresses are deliberately not signed so that NAT rebinding works.

5.4 Encryption status

rUDP version 2 provides **integrity and authentication, not confidentiality**. The source code says so in `RudpAuth`: "Ed25519 signatures provide integrity, NOT encryption or forward secrecy." Concretely:

- No cipher is used anywhere in IceBridge: no AES, ChaCha20, or key agreement. The X25519 keys in identity records are generated and published but never used. The labels `frostwire-icebridge-session-v1`, `frostwire-relay-transport-v1` and `frostwire-search-v1` are defined and referenced by no code.
- Every rUDP payload, every relay frame, and every application message travels in clear text. That includes search keywords, result rows, catalog pages and `.torrent` bytes. A network observer or a relay node can read them.
- What the signatures do provide: a packet cannot be injected, altered, replayed into another association, or reflected between directions without detection. Application messages carry their own end-to-end signatures, so a relay cannot forge a search result, catalog page or metadata chunk.
- There is no forward secrecy, because there are no session keys.
- The control API is plain HTTP on the loopback interface, protected by a bearer token.

5.5 Reliability

Each association keeps one outgoing sequence counter starting at 1 and one "received through" counter starting at 0. Sequence numbers are compared as unsigned 32-bit integers. Sequence 0 is reserved for handshake, acknowledgement and path packets. When the counter would wrap past $2^{32} - 1$ to 0, the association must be re-established.

- **In-order only.** A receiver accepts a sequenced packet only if it is exactly received-through + 1. It drops later packets (gaps) without buffering them. For a duplicate or old packet it resends a `DATA_ACK`. The protocol is effectively go-back-N.
- **Acknowledgement.** After the application accepts a packet, the receiver advances its counter and sends `DATA_ACK` (sequence 0, empty body, ack-through = received-through). Data packets also carry ack-through. An ACK means the next hop has taken ownership of the payload under its bounded queues. It does not mean the remote application processed it.
- **Backpressure.** If the application queue refuses a payload, the packet is not acknowledged, and the sender retransmits it later.
- **Window.** At most 32 unacknowledged sequenced packets are in flight per association (`SEND_WINDOW`).
- **Retransmission.** A timer runs every 500 ms. A packet is resent when $500 \cdot 2^{\min(r,3)}$ ms have passed since it was last sent, where r is its retry count. The schedule is 500, 1000, 2000, 4000,

4000 ms. If any pending packet reaches 5 retries, or 15 s since it was first sent, the *whole association* is dropped. The application must retry at its own level.

- **Idle timeout.** An authenticated association is dropped after 120 s without activity; an unauthenticated one after 15 s.

5.6 Fragmentation

A payload larger than 1024 bytes (`MAX_FRAGMENT_PAYLOAD`) is split into $k = \lceil n/1024 \rceil$ fragments. Each fragment is a sequenced packet whose body is:

Offset	Size	Field
0	4	group ID (random i32, per payload)
4	4	fragment index, 0-based
8	4	fragment total k
12	$\leq$1024	fragment data

Fragments $0..k-2$ have type `DATA_FRAG` and the last has type `DATA_END`. The receiver requires fragments in index order and allows no other data type while a group is open. Limits: $k \leq 256$; assembled size at most 256 KiB (262,144 bytes); at most 64 groups pending per node; a group older than 30 s ends the association. With the 12-byte header and the 64-byte signature, the largest protected payload is 1100 bytes, and the largest datagram is 1122 bytes, well under a 1500-byte MTU.

5.7 Path migration

When an authenticated packet with a valid signature arrives from a new address (for example after a NAT rebinding on a phone), the receiver does not process it. Instead it sends a protected `PATH_CHALLENGE` with 32 random bytes to the new address. If a protected `PATH_RESPONSE` with the same bytes comes back from that address within 5 s, the association moves to the new address. At most one challenge is outstanding per association every 5 s. A replayed packet therefore can never move an association by itself.

5.8 Admission control

Checks are ordered so that cheap checks come before signature verification:

- `HELLO`/`HELLO_ACK`: a global token bucket (burst 200, 50/s) and then a per-source-IP bucket (burst 10, 2/s, at most 4096 IP buckets). Outbound dials share the global bucket.
- Other packets: a global bucket of 2000/s (burst 2000) before signature verification.
- Associations: at most $\min(\text{maxSessions}, 4096)$ in total, where `maxSessions` defaults to 1024. At most 256 per IPv4 /24 or IPv6 /64.
- Retained data: per association 512 packets and 1 MiB; per node 16,384 packets and 16 MiB. Received datagrams pass through one FIFO worker with a 4096-entry queue. When the queue is full, the newest datagram is dropped and rUDP retransmission recovers it.

5.9 NAT traversal

Version 2 has *no hole-punching protocol*. The `HOLE_PUNCH` types exist but are dropped on receipt, because there is no authenticated way yet for a target to consent to an introduction. Earlier changelog entries (7.1.0) describe `HOLE_PUNCH` as implemented. That is no longer true. Nodes behind NAT are reached in three ways:

1. The NAT-ed node dials out to ultrapeers. The resulting association keeps the NAT mapping open. Leaves ping their uplinks at least every 30 s through `PeerRegistrySync`, which is shorter than the 120 s idle timeout.
2. Messages for a NAT-ed node go through a relay that already has an association with it (Section 5.10).
3. Two NAT-ed nodes that dial each other at the same moment may connect through the simultaneous-open rule. Nothing coordinates such an attempt.

On a LAN, nodes find each other's private addresses through multicast (Section 6.6), because most routers do not support hairpin connections to their own public address.

5.10 Relay frame

A `RELAY` packet body is a relay frame:

Offset	Size	Field
0	32	source pub: the *sending hop's* key
32	32	target pub: the final recipient's key
64	1	hop TTL (u8)
65	1..959	application payload (`MAX_APP_PAYLOAD` = 1024 − 65)

Relay frames are never fragmented, so a relayed payload is at most 959 bytes including the 6-byte IBP1 envelope. Delivery works as follows (`RudpSessionManager.deliver`, `handleRelay`, `forward`):

1. The sender delivers directly when it can: to itself, to a co-located client through the local poll queue, over a live association, or by dialling the target's registered address.
2. If that fails and the payload is at most 959 bytes, the sender *floods*: it sends a `RELAY` frame with TTL = `meshHopTtl` (3) to up to `meshBroadcastFanout` forwarders from its registry. The forwarders are shuffled, with at most 2 per IPv4 /24.
3. A forwarder accepts a frame only if its source pub equals the authenticated key of the association it arrived on, and its TTL is at most 6.
4. If the target is the forwarder itself, it delivers locally. If the target is in its registry, it sends a `RELAY_RESPONSE` to the target, with body `source pub (32) || payload`, over an existing or newly dialled association.
5. Otherwise, if TTL > 0, it forwards a new frame with the TTL reduced by 1 to its own forwarders, excluding the hop it came from. The new frame carries the forwarder's own key as source.

The recipient's application sees the *last hop's* key as the transport source of a relayed message. The original sender's identity is carried and verified only inside the signed application message. Flooding keeps no duplicate-suppression cache. TTL and the exclusion of the previous hop bound it, and applications drop duplicates through their own nonces and replay caches. The worst case is about $N + N^2 + N^3$ frames for fan-out N and TTL 3, so forwarders cap their fan-out (Table 9).

6 Discovery and the DHT

6.1 Why the DHT and IceBridge fit together

The BitTorrent Mainline DHT has millions of nodes, has no owner, and every BitTorrent client already runs it. It is good at two things: storing small signed values under a public key (BEP 44),

and remembering which `ip:port` pairs announced themselves under a 20-byte key (BEP 5). It cannot carry conversations, answer keyword queries, or reach a node behind NAT.

IceBridge is good at exactly those things. It does not have its own directory and does not need one. The split is:

	BitTorrent DHT (directory)	IceBridge (transport and services)
Stores	signed identity records, catalog manifests, karma chains (BEP 44)	nothing durable; peer registry and host cache in memory or on local disk
Finds	endpoints on the bootstrap, relay, peer and heartbeat topics (BEP 5)	the key behind an endpoint (rUDP handshake), then routes by key
Carries	at most 1000 bytes per item, no messages	reliable sessions, payloads up to 256 KiB, multi-hop relay
Trust	item signatures by the owner's key	the same key: sessions, messages and results are signed with it
Reach	public addresses only	NAT-ed nodes through outbound sessions and relays

The same Ed25519 key is the DHT item key, the rUDP session identity, and the signer of every search result. A node that finds an endpoint on a DHT topic proves who is behind it with an rUDP handshake, then looks up that key's signed identity record on the DHT to learn its role, capabilities and version. A user who receives a signed result from a key can fetch that key's catalog manifest and karma chain from the DHT, without contacting the node. Neither half needs a server that FrostWire runs.

6.2 BEP 44 mutable items

FrostWire publishes three items per identity. The code and older documents call them "BEP 46" items. They are BEP 44 mutable items with a salt. No BEP 46 info-hash pointers are used.

Item	Key	Salt (US-ASCII)	Value
Identity	Ed25519 pub	`frostwire-identity-v1`	signed identity record, `sig` included (Table 2)
Index manifest	Ed25519 pub	`frostwire-index-v1`	catalog manifest (Section 6.7)
Karma chain	Ed25519 pub	`frostwire-karma-v1`, `frostwire-karma-v1/<n>`	karma head page, older pages (Section 10)

Table 6: BEP 44 mutable items.

As BEP 44 specifies, the item target is SHA1(pub ‖ salt), and the item signature covers `4:salt<len>:<salt>3:seqi<seq>e1:v<bencoded value>`. Puts go through libtorrent's `dht_put_item` with a callback. libtorrent first fetches the current item, then the callback sets the value, increments the sequence number and signs. The sequence number is therefore the highest one seen plus one, or 1 for a new item. The application never chooses it. The 64-byte secret key passed to libtorrent is the expanded Ed25519 key, SHA-512 of the seed after clamping, not `seed || pub`.

Gets use `dht_get_item` with a timeout in whole seconds (5 s for identity, index and karma). The Java binding returns the *first* matching item alert. libtorrent can post a non-authoritative alert before the lookup finishes, so the result may not have the highest sequence number. A new implementation should wait for the authoritative result.

Republish intervals: identity at most every 300 s; index manifest when it changes or every 300 s; karma pages every 1800 s and right after each endorsement.

6.3 BEP 5 topics

A topic's info-hash is SHA1 of the topic's US-ASCII bytes.

Topic string	Who announces
`frostwire-bootstrap-v1`	dedicated servers started with `--bootstrap` (default for `FORWARDER` via environment). Looked up first.
`frostwire-relays-v1`	role `FORWARDER` or `BOTH`; a `CLIENT` only if marked connectable (no code marks it today)
`frostwire-peers-v1`	every desktop, Android nodes with an embedded server, and standalone daemons whose role is not `FORWARDER`
`frostwire-heartbeat-v1-<h>`	every node; $h = \lfloor \text{epochMillis}/3\,600\,000 \rfloor$ in decimal (the hour number). Used by crawlers to count presence.

Table 7: BEP 5 announce topics.

Announces use `dht_announce` with flags 0: no `implied_port`, not a seed. The announced port is the node's *rUDP* port, or the legacy port if the rUDP port is unknown. A node must not announce port 0. Desktop nodes announce every 30 s, Android every 300 s, standalone daemons every 60 s.

The desktop child process runs with `--no-dht`, and the parent announces through its BitTorrent session. A standalone daemon starts its own libtorrent session with only the DHT, on an ephemeral UDP port separate from rUDP, using the routers `dht.libtorrent.org:25401`, `router.bittorrent.com:6881`, `dht.transmissionbt.com:6881` and `router.silotis.us:6881`. If the DHT fails to start, the daemon keeps running without it.

Lookups use `dht_get_peers`. The Java binding returns after the first reply alert, that is, the first responding DHT node's peer list. A new implementation should collect replies from all nodes.

6.4 Discovery pass

A discovery pass runs every 30 s on desktop and every 300 s on Android. It gathers candidate `ip:rudpPort` endpoints from three sources, in order:

1. **Host cache and seeds** (preferred). The local host cache (Section 6.5) and the built-in seed `virginia1.frostwire.com:6889`. The system property `frostwire.icebridge.seeds` replaces the seeds with a comma-separated list; an empty value disables them.
2. **LAN beacon** (preferred). Private IPv4 senders heard on the multicast group.
3. **DHT**, with a 5 s budget: `frostwire-bootstrap-v1` for up to 2 s (results preferred), then `frostwire-relays-v1` for up to 3 s, then `frostwire-peers-v1` for what remains. Deduplicated by `host:port`, at most 64 candidates.

The pass drops loopback, link-local, multicast and own-interface addresses, our own public address on our own rUDP port, and endpoints in back-off. It orders preferred candidates first and shuffles the rest, takes at most 64, and probes them in parallel: 16 at a time, 3 s per probe, 15 s per pass.

A **probe** is an rUDP identity-discovery handshake (zero intended responder). The daemon exposes it as `POST /probe` and the caller polls it every 100 ms. When the handshake completes, the prober fetches the key's identity record from the DHT to learn its capabilities and version, then registers the peer as verified. If our own key answers, that endpoint is muted for 10 minutes. Failed endpoints back off from 30 s, doubling up to 10 minutes. Preferred endpoints back off from 5 s, doubling up to 60 s.

6.5 Host cache

The host cache is a UTF-8 text file at `~/.frostwire/icebridge_host_cache.txt` (on Android, under the app's `libtorrent` directory). It begins with a `#` comment line, and each further line is `host:port,ROLE,lastSuccessfulPingMs,consecutiveFailures`. The port is the rUDP port. The host is everything before the last colon, so IPv6 addresses are stored without brackets. Ultrapeers that answer a probe are added. An entry is removed after 5 consecutive failures. There is no size cap. Writes go to a temporary file that is then renamed, and happen at most once every 5 minutes, plus at shutdown.

6.6 LAN multicast beacon

Every 10 s a node sends the ASCII text `"FWIB1 <rudpPort>"` to `239.255.70.88:6890` on each active, multicast-capable, non-loopback, non-point-to-point IPv4 interface. It listens on `0.0.0.0:6890` with `SO_REUSEADDR`. A receiver accepts a datagram only if: it is at most 64 bytes; it is `FWIB1`, a space, and 1–5 digits forming a port from 1 to 65535; and the sender is an RFC 1918 private IPv4 address. Candidates expire after 120 s, with at most 128 kept. The beacon is not signed. It only produces candidates, and the rUDP handshake authenticates them.

6.7 DHT index manifest

A node that shares its catalog (Section 9) publishes a manifest of its public torrents under salt `frostwire-index-v1`:

```
{ "pub": <std Base64 of pub, no padding>,
  "rows": [ {"fc": <file count>, "ih": <hex info-hash>, "n": <name>, "s": <size>},
      ... ],
  "ts": <Unix seconds>, "v": 1 }                          (bencoded, keys sorted)
```

Rows are sorted by most recently seen. They are added while the bencoded manifest stays within 950 bytes, so the manifest holds only the first few torrents. The full catalog is available through mesh browsing (Section 8.2). The manifest has no signature of its own. Only the BEP 44 item signature protects it. When the user stops sharing the catalog, or nothing is public, an empty `rows` list replaces the previous manifest.

7 Relay mesh, topology and control API

7.1 The IBP1 application envelope

Every payload the application hands to IceBridge is wrapped in a 6-byte envelope:

Offset	Size	Field
0	4	magic `"IBP1"` (`49 42 50 31`)
4	2	protocol ID, u16 big-endian
6	≥ 1	payload (to the end of the rUDP payload; no length field)

When wrapping, protocol ID 0 is changed to 1. When unwrapping, the raw ID is returned. A received ID 0 is treated as `SEARCH` by the application. Unknown IDs are delivered and logged, never rejected. This makes the transport independent of content: a new service only needs a new ID.

7.2 Topology

IceBridge borrows the two-tier shape of Gnutella with ultrapeers. Ultrapeers keep associations with many other ultrapeers and leaves. Leaves keep a few uplinks. The values below are what runs:

ID	Name	Use
0	UNSPECIFIED	treated as SEARCH
1	SEARCH	distributed search request and response (Section 8.1)
2	CHAT	reserved
3	METADATA	TORRENT_FETCH request and chunks (Section 8.3)
4	PUBSUB	reserved
5	AI	reserved
6	TELEMETRY	ping: the single byte 0x01
7	FILESYNC	reserved
8	INDEX_DIGEST	keyword Bloom filter of a node's index
9	NODE_META	capability mask: [0x01][caps i64 BE], exactly 9 bytes
10	CLUSTER_DIGEST	union of an ultrapeer's own and its leaves' digests
11	CATALOG	catalog browse request and pages (Section 8.2)

Table 8: Mesh protocol IDs (MeshProtocolId).

IceBridgeConfig.fromEnv applies a per-role profile on top of the compiled defaults. Environment variables ICEBRIDGE_MESH_FANOUT, ICEBRIDGE_SEARCH_PEER_FANOUT, ICEBRIDGE_MESH_HOP_TTL, ICEBRIDGE_SEARCH_TTL, ICEBRIDGE_SOFT_MAX and ICEBRIDGE_LEAF_UP_CONNECTIONS override them.

Parameter	Compiled default	Ultrapeer profile	Leaf profile	Hard max
mesh broadcast fan-out N	6	32	6	96
search peer fan-out M	8	32	6	96
mesh hop TTL	3	3	3	6
search TTL	2	7	7	7
soft max (hops + TTL)	3	7	7	7
leaf uplinks	3	6	6	16

Table 9: Topology parameters (IceBridgeTopology). The standalone daemon, the desktop and Android apply the role profile; a server built in code that does not apply it keeps the compiled defaults.

Peer registry sync. On a FrostWire node, PeerRegistrySync runs every 30 s and keeps the daemon's registry and the application's peer directory consistent:

1. it registers its own identity with the daemon (signed /register);
2. it pushes up to 100 verified peers, ranked by trust, as routes;
3. it pulls up to 100 routes from /lookup, ignoring entries more than 10 minutes old;
4. it pings uplinks (TELEMETRY 0x01): 32 ultrapeers if it is one, otherwise 6;
5. it sends its INDEX_DIGEST to 6 ultrapeers when the digest changes, or at least every 120 s;
6. an ultrapeer sends a CLUSTER_DIGEST to 32 ultrapeers at most every 120 s;
7. it sends NODE_META to 16 ultrapeers at most every 120 s.

Keyword digests. An INDEX_DIGEST payload is 0x01 followed by a Bloom filter whose length is a power of two between 64 and 16,384 bytes (4096 for a leaf, 16,384 for a cluster), with $k = 4$ hash functions. For hash $i \in \{0..3\}$: h_i is the first 8 bytes, read big-endian, of SHA256(byte(i) $\|$ UTF8($token$)). The bit index is the top $\log_2(bits)$ bits of h_i, set as byte[b»3] |= 1 « (b & 7). Tokens come from NFD normalization, removal of combining marks, lower-casing with Locale.ROOT, and runs of ASCII [a-z0-9] of length at least 3, minus stop words. At most 2048 tokens are taken from at

most 16,384 input characters. A query of n tokens matches if at least n tokens hit when $n \leq 2$, otherwise at least $\lfloor (2n+2)/3 \rfloor$. Digests let searchers and forwarders choose likely holders first. A false positive costs one extra query, and a false negative cannot happen for indexed tokens. Note that the tokenizer is ASCII-only, so names in non-Latin scripts produce no tokens and gain nothing from digests.

7.3 Control API

The application talks to its IceBridge daemon over HTTP/1.1 on `127.0.0.1`. The desktop launches the child with `--control-http-port 0`. The child prints a line `ICEBRIDGE_CONTROL_PORT=<port>` on standard output, and the parent reads it. There is no announcement file. Every endpoint except `/health` requires the header `X-IceBridge-Token`. The parent generates the token as 32 random bytes in lowercase hex and hands it over in an owner-only file (`--auth-tokens-file`), never on the command line. Tokens are compared in constant time. Request bodies are JSON and at most 64 KiB. Responses are `{"ok":true,"data":...}` or `{"ok":false,"error":"..."}`. Binary values are Base64url without padding. Unknown endpoints return HTTP 400; there is no 404.

Endpoint	Purpose
`GET /health`	liveness; no token
`POST /register`	signed self-registration: `{pub, host, rudpPort, role, timestamp, icebridgeVersion, signature}`. The signature is Ed25519 over the UTF-8 bytes of `len(pub):pub|len(host):host|rudpPort|ROLE|timestamp` plus `|len(v):v` if a version is given, where `len` counts UTF-16 units and `pub` is the Base64url string. Timestamp within ±60 s.
`POST /route`	unsigned route hint `{pub, host, rudpPort, role, ifAbsent}`
`POST /consumer`	subscribe or unsubscribe a per-identity inbound queue
`POST /probe`	rUDP identity discovery of `{host, port, cancel}`; returns `none`, `pending` or `established` with `pub`
`GET /lookup`	up to 100 registry entries `{pub, host, rudpPort, role, lastSeenMs, icebridgeVersion}`
`POST /send`	`{targetPub, payload, protocolId}`: wrap in IBP1 and deliver
`GET /poll`	drain up to 256 messages (256 KiB) `{sourcePub, payload, receivedMs, protocolId}`; not long-polling; messages are removed when returned
`GET /metrics`	cumulative counters (packets, bytes, rejections, registry size)
`GET /events`	recent event log (ring buffer of 5000)
`GET /catalog`	browse a peer's catalog (timeout 100–30,000 ms, default 5000)
`GET /torrent`	`TORRENT_FETCH` by info-hash from a holder (default 15,000 ms)

Inbound queues hold at most 512 messages each, 256 consumers, 4096 messages and 16 MiB in total. The desktop polls every 300 ms. The parent supervises the child: it checks `/health` every 10 s, restarts the child on failure, and keeps the same rUDP port across restarts. The child exits when the parent's PID disappears (checked every 3 s).

8 Services

All service messages are UTF-8 JSON objects, produced by Gson. Method names in the code say "bencode", but the wire format is JSON. Binary fields use *standard* Base64 without padding, unless noted. Hex is written in lowercase and read in either case. Timestamps are Unix seconds and must be within ±300 s of the receiver's clock. Signatures are pure Ed25519 by the sender's identity key

over a byte string built as described, not over the JSON. In the signature layouts below, `i32`/`i64` are big-endian and `lp(x)` means `i32 length || x`.

8.1 Distributed search (protocol 1)

Request (`RemoteSearchRequest`, version 3):

```
{"v":3, "k":<keywords>, "lim":<1..100>, "nonce":<32 bytes>, "ttl":<int>, "maxTtl":<
    int>,
 "pub":<requester pub>, "path":[<pub>, ...], "ts":<seconds>, "sig":<64 bytes>}
```

Signed bytes:

```
i32 v | lp(UTF-8 keywords) | i32 lim | lp(nonce) | lp(requester pub) | i64 ts | i32
     maxTtl
```

`ttl` and `path` are not signed, so forwarders can change them within the signed `maxTtl`. A receiver checks: all fields present; `v = 3`; keywords at most 1024 UTF-8 bytes; `0 < lim <= 100`; `path` has at most 8 unique 32-byte keys; $0 \le ttl$, $maxTtl \le 8$, $|path| \le maxTtl$, and $ttl \le maxTtl - |path|$. Keywords are sent as typed. Each responder normalizes them for its own index.

Originating a search. The searcher picks peers from its directory: verified peers ranked by trust, nodes with `SEARCH` capability preferred (ultrapeers when the searcher is a leaf), then ranked by XOR distance between SHA1(UTF8(lowercase(trim(*keywords*)))) and the first 20 bytes of each peer's key, then nodes whose digest matches are moved first. It keeps at most M peers, with at least one ultrapeer. It sends each peer its own request with a fresh 32-byte nonce, `lim = 25`, $ttl = \min(7, searchTtl)$, $maxTtl = ttl + 1$ and `path = [own pub]`. The desktop waits up to 10 s and stops early once it has 200 rows. The code also contains a phased "dynamic query" mode, used only by callers that ask for it: up to 3 phases with 4, 12, then all peers, TTL 1, 2, then full, and timeouts of 3, 6 and 10 s, stopping at 10 unique results.

Answering. The responder (`RelaySearchService`) checks, in order: $0 < ttl \le 8$ and it is not in the path; the requester is not marked as spam; the timestamp; a global ingress limit (burst 100, 100/s); the signature; replay of `(requester, nonce)` (4096 entries, 601 s); a per-requester limit (burst 5, 5/s). It then searches its local index for $\min(lim, 100)$ rows. The index is SQLite FTS with the `porter unicode61` tokenizer and bm25 ranking. The query is lower-cased with `Locale.ROOT`, characters other than letters and digits become spaces, and tokens are quoted and joined with AND. Results are filtered by the share-visibility policy (Section 9). A node with a local index answers even when it has no matches, with a signed empty final response. A pure forwarder (no index) never sends an empty response, because an empty final would end the requester's wait for that peer before the holders behind it answer. If any row loses visibility while the answer is built, the node sends nothing.

Forwarding. If $ttl > 1$ and the node is not a `CLIENT`, it forwards the request at most once per request. It does not re-sign: it appends its own key to `path` and sets $ttl' = \text{clamp}(|path|, ttl - 1)$, which limits hops plus remaining TTL to the soft maximum. Targets, up to M in total: $\max(1, M/4)$ peers whose digest matches, then ultrapeers whose cluster digest allows the query, then any verified peer. Path members, the source, the requester and itself are excluded. Responses go directly to the requester's key, not back along the path.

Response (`RemoteSearchResponse`, version 4):

```
{"v":4, "nonce":<request nonce>, "ts":<seconds>,
 "rows":[{"ih":<hex 20>, "n":<name>, "s":<size>, "fc":<file count>, "pub":<holder
     pub>,
          "nid":<hex 20>?, "mf":<matched file>?, "bt":[<host:port>,...]?, "pc":true
              ?}, ...],
 "chunk":<index>, "final":<bool>, "sig":<64 bytes>}
```

Signed bytes:

```
i32 v | lp(nonce) | i64 ts | SHA-256(rowsBlob) | i32 chunk | u8 final
```

`rowsBlob` is the concatenation, in row order, of one bencoded dictionary per row. The keys are sorted (`bt, fc, ih, mf, n, nid, pc, pub, s`), optional keys appear only when present, and *every value is a bencode string*: numbers in decimal, `bt` joined with commas, `ih` and `nid` in lowercase hex, `pub` in standard Base64 without padding, and `pc` as `"1"`. `bt` lists up to 8 of the holder's BitTorrent endpoints, so the downloader can connect to the holder directly. `pc` says the holder shares its catalog, which means the result can be browsed.

Up to 5 rows go in one frame (`chunk 0, final true`). Larger answers are split into chunks of 5 rows, with the same nonce and timestamp, each signed separately, the last marked final. The requester accepts at most 128 chunks and 256 KiB per holder. It verifies each frame: the nonce matches its request, the timestamp is fresh, and the signature verifies under the key it sent the request to, or else under the single holder key shared by all rows. Results become magnet links extended with `x.pe` (the `bt` endpoints), `x.hp` (the holder's key, Base64url) and `x.hc=1` (the result can be browsed).

8.2 Catalog browsing (protocol 11)

A user can list everything a peer shares, if that peer allows it.

Request: `{"v":1, "pub", "target", "nonce", "ts", "sig"}`, signed over `i32 v | lp(pub) | lp(target) | lp(nonce) | i64 ts` (120 bytes). The holder answers only if catalog sharing is enabled, `target` is its own key, the nonce is 32 bytes, the timestamp is fresh, the ingress and per-requester limits allow it (burst 30, 0.5/s, shared with metadata requests), and the request is not a replay.

Pages (`CatalogManifestPages`, version 2): the holder lists at most 100 torrents, most recently seen first, that pass the catalog visibility policy. Names longer than 180 UTF-16 units are cut to 179 and `...` is appended. It splits them into at most 64 pages of at most 937 bytes of rows each, so that each page fits one relay frame:

```
{"v":2, "pub":<std b64>, "nonce":<std b64 WITH padding>, "page":i, "pages":n,
 "rows":[{"ih","n","s","fc"}, ...], "ts", "sig"}
```

Signed bytes:

```
i32 2 | lp(ASCII of the pub Base64 text) | i64 ts
      | SHA-256( for each row: lp(ih hex) | lp(UTF-8 name) | i64 size | i32 file
          count )
      | nonce (32) | i32 page | i32 pages
```

The signature covers the `pub` Base64 text exactly as sent. Current holders write it without padding; the test fixtures pad it. Verify over the received text, do not re-encode it. The browser checks that `pub` equals the target, that the nonce matches, and the signature. It returns what it has when the timeout expires (8 s in the UI). It does not check the timestamp. When mesh browsing fails, the UI falls back to the DHT index manifest (Section 6.7).

8.3 Torrent metadata fetch (protocol 3, TORRENT_FETCH)

A searcher that has a result but cannot get the torrent's metadata from the swarm can fetch the `.torrent` file from the holder.

Request: `{"v":1, "ih":<hex 20>, "nonce", "pub", "ts", "sig"}`, signed over `i32 v | i32 20 | ih | lp(nonce) | lp(pub) | i64 ts`.

Holder: it checks the nonce, timestamp, limits, signature and replay, as for catalog requests. If the torrent is not public under the search policy, it replies with a signed `NOT_FOUND`. Otherwise it charges a global work budget (burst 1 MiB, 256 KiB/s) and drops the request silently if the budget is exhausted. It replies with `TOO_LARGE` above 256 KiB. Otherwise it splits the `.torrent` bytes into chunks of 448 bytes, signs each, and caches the signed chunks (32 torrents, 5 minutes).

Chunk (`TorrentMetadataResponse`, version 3): `{"v":3, "nonce", "ih", "pd":<SHA-256 of the whole .torrent>, "ci":<chunk index>, "fin":<bool>, "ts", "data"|"err", "sig"}`, with at most 586 chunks. Signed bytes:

```
"FW-METADATA-3" | i32 v | i32 20 | ih | pd (32) | i32 ci | u8 fin | u8 isErr
                | [ lp(nonce) | i64 ts ]      (error frames only)
```

Data chunks do not sign the nonce or timestamp, so the holder can cache and resend them. `pd` binds every chunk to the whole file. The requester checks the nonce and info-hash, verifies each chunk under the holder's key, reassembles, checks `pd`, and libtorrent checks the info-hash. At most 8 fetches run at once (2 per holder); the default timeout is 15 s.

9 Seeding is publishing: the consent model

FrostWire treats sharing a torrent in the swarm as the act of publishing it. There is no separate "upload to index" step, and nothing that is not active in the user's BitTorrent session is ever offered. The code has two levels of exposure:

1. **Keyword search** (joining the network, setting `ICEBRIDGE_ENABLED`, default on). Anyone whose query matches can find a torrent that is *active* in the session. On desktop (`BtTransferShareVisibility`) a torrent is active when it is live, has valid metadata, is not paused, is not private, and its state is `SEEDING` *or* `DOWNLOADING`. A downloading torrent already uploads pieces to the swarm, so it is shared in the BitTorrent sense. On Android the search policy does not check the state: any live, unpaused, non-private transfer with metadata is served. Paused, removed, finished-and-stopped, and private torrents are never served. The same policy applies to `TORRENT_FETCH`.
2. **Catalog listing** (setting `ICEBRIDGE_PUBLIC_CATALOG`, default *on*, that is, opt-out). Browsing a node's whole catalog, and publishing the DHT index manifest that crawlers read, require this setting *and* the state `SEEDING`. When the setting is off at startup, the node ignores catalog requests, publishes an empty manifest in place of the previous one, and clears `PUBLIC_CATALOG` from its capabilities. Turning it off while FrostWire runs takes full effect after a restart; until then the node answers catalog requests with a signed empty page. Catalog sharing cannot be enabled while the node is off the network.

Consequences:

- Stop seeding (pause or remove) and the torrent disappears from answers at once. The policy is checked when each answer is built, not when the index is written.

- Results are fresh: every result names a holder that was sharing the torrent seconds ago, and `bt` lists that holder's endpoints.
- Private torrents (BEP 27) are never published.
- Crawler-facing data identifies publishers only by public key, never by IP address (`x.hp`).
- Keyword search cannot be disabled separately from network membership. A user who wants to search must answer searches.

10 Karma and reputation

Each identity keeps an append-only, hash-linked *karma chain* of signed entries. Entries are anchored to Bitcoin block heights so that their age cannot be faked forward.

10.1 Entries

There are two kinds. An *epoch commitment* (`EC`) opens an epoch of 144 Bitcoin blocks (about one day) and states the energy available for endorsements. An *endorsement* (`EN`) records that the owner downloaded torrent `ih` from peer `pp`, found through distributed search. An endorsement is written automatically when such a download finishes or starts seeding: once per peer and torrent, only for v1 info-hashes, and never for one of the owner's own current or earlier keys.

Signed bytes are the canonical bencoding of:

```
common: bh  <block height>          bk  <hex block hash, 64 chars>
        k   "EC" | "EN"             ph  <hex prev hash, 64 chars>
        pub <Base64url of owner pub, no padding>
        seq <0-based index>
EC:     e   <energy formatted "%.2f" with Locale.ROOT>    ep <epoch = bh / 144>
EN:     ih  <hex info-hash, 40>     pp <Base64url of endorsed pub>   sd <score
    delta, 1>
```

The entry hash is SHA256(*canonical bytes*). The next entry's `ph` must equal it, and the first entry's `ph` is 32 zero bytes. The local creation time is not signed.

Energy. The first `EC` has energy 5. Each following `EC` carries $R = \max(0, E_{\text{prev}} - \textit{endorsements in previous epoch})$. In chain format 1 (FrostWire up to 7.1.3 and 3.2.3) its energy is $5 + 0.5R$, applied once however many epochs were skipped. In format 2, written since then, it is $5 + 0.5^{\Delta}R$, where Δ is the number of epochs since the previous `EC`; a format 2 `EC` signs `fv = 2` in its canonical bytes (format 1 has no `fv`). Each `EC` is checked against its own format's rule, and a chain never goes back from format 2 to format 1. Energy is capped at 10. An epoch allows $\lfloor E \rfloor$ endorsements.

Verification. A verifier checks: a single owner; valid signatures; hash links; `seq` equals the index; block heights never decrease; `ep = bh/144` and epochs strictly increase; energy within $[0, 10.01]$ and within 0.01 of the expected value; no self-endorsement; endorsements per epoch at most $\lfloor E \rfloor$. With a known Bitcoin tip (cached for 60 s), it also rejects heights above tip + 2 and checks each `EC` block hash against the real hash at that height.

10.2 Bitcoin anchoring

The writer anchors each entry to the current Bitcoin tip. It reads the height and block hash over HTTPS from `blockstream.info` and falls back to `mempool.space` (`/api/blocks/tip/height`, `/api/block-height/<h>`), with a 5 s timeout. Hashes are kept in the byte order the APIs display.

Headers are not downloaded and proof of work is not checked: the anchor trusts these two APIs. When they cannot be reached, the writer does not write, and the verifier skips the block checks.

10.3 Publication

Every 1800 s, and right after each endorsement, the node commits an epoch if needed and publishes its chain in pages (manifest version 2), each a BEP 44 item of at most 950 bytes. The head page uses salt `frostwire-karma-v1`; up to 7 older pages use `frostwire-karma-v1/<n>`, $n = 1, 2, \ldots$ counting back from the head. A page is `{v:2, seq, ph, e}`, and the head adds `len`, `head` and `pg` (the number of older pages). Its entries are compact: raw binary `bk` (block hash), `s` (signature), `pp` and `ih`; `en` is the signed `"%.2f"` text; `fv` when 2; `sd` only when not 1. Each entry's seq and prevHash follow from the page's `seq` and `ph` and the previous entry, its epoch from its height, and its owner from the item's key, so the signed canonical bytes are rebuilt unchanged. An endorsement takes about 196 bytes (362 in version 1). The head holds the newest entries that fit, starting at an `EC` when one is among them, so it verifies alone. Every non-empty chain is published.

A reader fetches the head, then older pages while each links to the page after it (seq and hash), within the lookup's 5 s deadline. It verifies the result from its first `EC` as a tail, or as the whole chain from the genesis, and falls back to the head alone if older pages don't verify. A head without `v:2` is a version 1 manifest (`{base?, entries, head, len, pub, ts, v:1}`, with hex and Base64url fields), which released clients still publish and readers still accept. Released clients read only the head item and treat a version 2 head as no chain. Readers accept at most 64 KiB and 256 entries.

10.4 Use in ranking

A peer's karma is the endorsements it has *received*, as this node sees them (`ReceivedKarma`). Every chain the node verifies is indexed by endorsee. An endorsement of X by E for torrent T counts only if E's identity has proof of work, $E \neq X$, and X announced T itself: this node verified a search response that X signed listing T as its own (or T is in this node's index, when X is this node). Each endorser casts one vote per endorsee, worth $0.5^{a/30}$ for the newest counting endorsement a epochs old. Votes are weighted by trust that starts at this node:

$$T(\mathit{self}) = 1, \quad T(X) = \min\Big(1, \sum_E T(E) \cdot 0.5 \cdot \mathit{vote}(E, X)\Big) \text{ (3 hops)},$$

$$w(E) = \max\big(T(E),\ 0.1 \text{ if this node first saw } E \text{ at least 7 epochs ago}\big),$$

$$\mathit{karma}(X) = \sum_E w(E) \cdot \mathit{vote}(E, X),$$

with $w(\mathit{self}) = 1$. The age is witnessed (first sightings are kept on disk), because an owner can sign entries at any past height. The endorsements a peer has *given* are shown separately as participation and never count as trust. Distributed search results show karma in a hidden-by-default Karma column, and sort by it after seeds. Peer selection prefers peers with higher trust, where trust is 1 plus karma plus local adjustments (a peer marked as spam gets -1).

10.5 Limitations in the current code

These follow from the code as written. They are listed so that a new implementation does not copy them by accident.

- Received karma is a partial, local view: an endorsement counts only once this node has verified the endorser's chain and seen the endorsee announce the torrent. Nothing yet answers "who endorsed X" for an arbitrary X (see `docs/karma-received-endorsements-proposal.md`).
- A reader sees at most the head and 7 older pages, about 30 endorsements of a chain.

- The transitive web-of-trust code (`PeerDirectory.addEndorser`) has no production caller, so the transitive term is always 1.
- Proof of work on identities is not checked for remote keys, so the Sybil cost is low.
- Nothing bounds a chain's starting height from below, so a new identity can create a chain that looks old. `EN` block hashes are not checked. An owner can fork or reset its own chain, because `head` and `len` are not checked.

11 Security and threat model

11.1 Assets and adversaries

The assets are: the integrity of search results (users download what results describe), the availability of the network, the user's control over what is published, and the local machine (the control API). The adversaries considered are: a network attacker who can see, drop, inject and replay UDP; malicious peers and relays; Sybil identities; and local processes on the same machine.

11.2 What the design resists

Forged or altered messages. rUDP packets are signed per packet with a transcript-bound context. Application messages are signed end to end, and relays cannot change them. `TORRENT_FETCH` binds chunks to the digest of the whole file, and libtorrent checks the info-hash.

Replay. `HELLO` timestamps must be within ±60 s. Every protected packet is bound to the transcript, CID, direction and sequence. Application requests carry 32-byte nonces and timestamps (±300 s), with replay caches of 601 s.

Session hijack by address spoofing. An association moves to a new address only after a signed path challenge succeeds from that address.

Amplification and floods. Cheap checks come before signature verification. HELLO admission is limited per IP and globally. Associations are capped globally and per /24. Relay floods are limited by TTL and fan-out. Search forwarding is limited to one forward per request. Every queue has a bound.

Local abuse of the control API. It listens on loopback only and requires a 256-bit token for everything except `/health`.

Unwanted publication. Only active, non-private torrents are served, and full catalog listing is a separate setting.

11.3 What it does not resist

Eavesdropping. Nothing is encrypted (Section 5.4). Anyone on the path, and every relay, can read queries and results. An observer can link a query to the requester's IP address and key.

Traffic analysis and deanonymization. Endpoints are public on the DHT. Results carry the holder's BitTorrent endpoints.

Sybil attacks. Identities cost nothing for remote verifiers. Karma can be inflated (Section 10). An attacker with many identities can fill DHT topics with endpoints and fill peer directories.

Malicious content. A signature proves who shared a torrent, not that the torrent is what its name says. Karma is the intended defence, and it is weak today.

Search-path tampering. `ttl` and `path` are not signed. A forwarder can shorten or lengthen propagation within the signed `maxTtl`.

Unauthenticated hints. `/route` and the LAN beacon are unauthenticated. They only create candidates or routes that the rUDP handshake must still confirm, but a peer imported through `/lookup` is marked verified without a local handshake.

Clock dependence. Nodes with skewed clocks fail handshakes and requests.

Bitcoin anchoring trust. Anchors trust two HTTPS APIs and do not handle reorganizations.

12 Implementation notes for new implementations

The following points are easy to get wrong when writing an implementation in another language, such as Rust. The Java code defines the behaviour.

12.1 Encodings

- **Two Base64 alphabets.** The control API and karma use Base64url without padding. Identity records, search, catalog and the DHT index manifest use *standard* Base64 without padding, except the catalog page `nonce`, which is padded. Comments in the code that say "base64url" for search payloads are wrong. Java's decoders accept missing padding and non-zero trailing bits; strict Rust decoders reject the latter.
- **Canonical bencode** for every signed dictionary: keys sorted by raw bytes, shortest integers, lowercase hex, no unknown keys in the signed set.
- **Re-encode, then verify.** Identity records and karma entries are verified over bytes rebuilt from parsed fields. An implementation that verifies the received bytes accepts and rejects different inputs.
- **Lengths in UTF-16 units.** Name caps (2048, 180), host length in `/register` and the `/register` signing string use Java `String.length()`. Keyword caps use UTF-8 bytes. Truncating to 179 units can split a surrogate pair. Java then encodes the lone surrogate as `?` in UTF-8.
- **JSON numbers** pass through IEEE double in Gson, so integers above 2^{53} lose precision. Do not rely on them.
- **Java `trim()`** removes characters $\leq$ U+0020 only. `toLowerCase(Locale.ROOT)` is required wherever the code uses it; the Turkish dotless i caused bug #1407.
- **Karma energy formatting.** Java's `"%.2f"` rounds half-up on the shortest decimal representation: 9.125 becomes `"9.13"`, where C and Rust give `"9.12"`. Energies such as 9.125 occur. Also, the verifier signs `"%.2f"` of `parseDouble("%.3f"(x))`, which can differ from what the author signed (for example 9.0048828125). Match the Java behaviour exactly. Manifest version 2 carries the signed `"%.2f"` text itself, so it has no double rounding.

12.2 Protocol behaviour

- rUDP accepts only in-order packets. Do not add selective ACKs to version 2: a Java peer will not understand them.
- Treat the transport source of a relayed message as the last hop. Authenticate the origin only through the signed application message.
- A relayed payload, including the 6-byte envelope, must fit 959 bytes. Larger payloads need a direct association.
- Send `HOLE_PUNCH` to a Java v2 peer and it is ignored.

- Use the bound rUDP port in identity records, `/register` and DHT announces. Never announce 0.
- Read DHT results the way libtorrent finishes them: wait for authoritative BEP 44 gets, and collect all `get_peers` replies. The Java binding does neither.
- Let libtorrent (or equivalent) choose BEP 44 sequence numbers as *highest seen + 1.*
- `/poll` is at-most-once: a crash after polling loses messages.
- Responses to a search are sent to the requester's key, not back along the path.

12.3 Compatibility checklist

An implementation that interoperates with FrostWire 7.1.3 for desktop and FrostWire 3.2.3 for Android must:

1. Derive Ed25519 keys from 32-byte seeds per RFC 8032, and export and import the seed as a 24-word BIP39 mnemonic with no PBKDF2.
2. Produce and verify identity records v1–v4 (Table 2) in canonical bencode, signature over the record without `sig`.
3. Put and get BEP 44 items with salts `frostwire-identity-v1`, `frostwire-index-v1` and `frostwire-karma-v1`, keyed by the identity key.
4. Announce on and query `SHA1("frostwire-bootstrap-v1")`, `SHA1("frostwire-relays-v1")`, `SHA1("frostwire-peers-v1")` and the hourly heartbeat topic, with the rUDP port.
5. Speak rUDP v2: 22-byte header with magic `0x4677`, version 2; the four-message handshake with 168- and 192-byte payloads; per-packet signatures with domain `FrostWire-IceBridge-rUDP-v2`; in-order reliability; 12-byte fragment headers; path challenges; relay frames with a 65-byte header.
6. Wrap application payloads in `IBP1` with the protocol IDs of Table 8.
7. Implement search request v3 and response v4, catalog request v1 and pages v2, metadata request v1 and chunks v3, with their exact signature inputs.
8. Send and accept `INDEX_DIGEST`, `CLUSTER_DIGEST`, `NODE_META` and `TELEMETRY` pings.
9. Optionally send and receive the LAN beacon `FWIB1 <port>` on `239.255.70.88:6890`.
10. Apply the share-visibility rules of Section 9.
11. Respect the limits in Appendix A, because Java peers enforce them and drop anything larger.

13 Status and future work

IceBridge shipped in FrostWire 7.1.0 for desktop, hardened in 7.1.1–7.1.3, and in FrostWire 3.2.0 for Android, hardened in 3.2.1–3.2.3. Desktops, phones and standalone forwarders run the same protocol and interoperate. Work that the code or documents identify as open:

- **Confidentiality.** End-to-end encryption of application payloads with the X25519 keys already in identity records, and transport encryption with forward secrecy. This changes the wire format and needs a new rUDP version.
- **NAT traversal.** An authenticated introduction protocol in which the target consents, so that `HOLE_PUNCH` can be re-enabled.
- **Karma.** Let any peer find the endorsements of any other (an endorsee-published "received" manifest, or gossip of endorsement references), populate the Peer Directory's web of trust, and verify Bitcoin headers instead of trusting HTTP APIs.

- **Connectivity detection.** `ConnectivityDetector.markConnectable` has no caller, so a `CLIENT` never promotes itself to the relay topic, and Android leaf promotion never triggers.
- **Search tokenization** for non-Latin scripts in keyword digests.
- **New services** on reserved protocol IDs: chat (2), pub/sub (4), AI (5) and file sync (7).
- **Growth.** Central index sites become less necessary as more peers join and share fresh content. Third-party crawlers can already index public catalogs.

References

[1] B. Cohen. *BEP 3: The BitTorrent Protocol Specification.* `https://www.bittorrent.org/beps/bep_0003.html`

[2] A. Loewenstern, A. Norberg. *BEP 5: DHT Protocol.* `https://www.bittorrent.org/beps/bep_0005.html`

[3] G. Hazel, A. Norberg. *BEP 9: Extension for Peers to Send Metadata Files.* `https://www.bittorrent.org/beps/bep_0009.html`

[4] D. Harrison. *BEP 27: Private Torrents.* `https://www.bittorrent.org/beps/bep_0027.html`

[5] A. Norberg, S. Siloti. *BEP 44: Storing arbitrary data in the DHT.* `https://www.bittorrent.org/beps/bep_0044.html`

[6] L. Garbis. *BEP 46: Updating Torrents Via DHT Mutable Items.* `https://www.bittorrent.org/beps/bep_0046.html`

[7] M. Palatinus, P. Rusnak, A. Voisine, S. Bowe. *BIP 39: Mnemonic code for generating deterministic keys.* `https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki`

[8] S. Josefsson, I. Liusvaara. *RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA).* IETF, 2017.

[9] A. Langley, M. Hamburg, S. Turner. *RFC 7748: Elliptic Curves for Security.* IETF, 2016.

[10] NIST. *FIPS 180-4: Secure Hash Standard (SHA-1, SHA-256, SHA-512).* 2015.

[11] S. Josefsson. *RFC 4648: The Base16, Base32, and Base64 Data Encodings.* IETF, 2006.

[12] T. Bray. *RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format.* IETF, 2017.

[13] B. H. Bloom. Space/time trade-offs in hash coding with allowable errors. *CACM* 13(7), 1970.

[14] A. Singla, C. Rohrs. *Ultrapeers: Another Step Towards Gnutella Scalability.* Gnutella Developer Forum, 2002.

[15] P. S. Efraimidis, P. G. Spirakis. Weighted random sampling with a reservoir. *Information Processing Letters* 97(5), 2006.

[16] A. Norberg et al. *libtorrent (Rasterbar).* `https://libtorrent.org`

[17] FrostWire. *frostwire-jlibtorrent.* `https://github.com/frostwire/frostwire-jlibtorrent`

A Wire formats and constants

A.1 Strings and identifiers

Value	Use
`0x4677`	rUDP magic (“Fw”)
`2`	rUDP protocol version
`FrostWire-IceBridge-rUDP-v2`	rUDP signature domain (27 bytes)
`IBP1`	application envelope magic
`FW-METADATA-3`	metadata chunk signature domain
`frostwire-identity-v1`	BEP 44 salt, identity record
`frostwire-index-v1`	BEP 44 salt, index manifest
`frostwire-karma-v1`	BEP 44 salt, karma chain head page
`frostwire-karma-v1/<n>`	BEP 44 salt, karma chain older page n (1–7)
`frostwire-bootstrap-v1`	BEP 5 topic
`frostwire-relays-v1`	BEP 5 topic
`frostwire-peers-v1`	BEP 5 topic
`frostwire-heartbeat-v1-<hour>`	BEP 5 topic, hourly
`FWIB1 <port>`	LAN beacon payload
`239.255.70.88:6890`	LAN beacon group and port
`virginia1.frostwire.com:6889`	built-in seed
`X-IceBridge-Token`	control API header
`ICEBRIDGE_CONTROL_PORT=<n>`	child stdout line with the control port
`1.1.0`	IceBridge software version (`ib_ver`)
`30 2a 30 05 06 03 2b 65 70 03 21 00`	X.509 prefix of an Ed25519 public key

A.2 Limits and timers

Item	Value
rUDP	
header / max wire payload / fragment data	22 / 1152 / 1024 bytes
HELLO / HELLO_ACK payload	168 / 192 bytes
HELLO clock skew	±60 s
send window	32 packets
retransmit tick, backoff	500 ms; $500 \cdot 2^{\min(r,3)}$ ms
max retries / retransmit timeout	5 / 15 s (whole association dropped)
idle timeout (authenticated / not)	120 s / 15 s
fragments per group / assembled size	256 / 256 KiB
pending fragment groups / group timeout	64 / 30 s
pending per association	512 packets, 1 MiB
pending per node	16,384 packets, 16 MiB
associations: total / per /24 or /64	$\min(1024, 4096)$ / 256
HELLO limits: global / per IP	200 burst, 50/s / 10 burst, 2/s
packet limit (global)	2000/s
ingest queue	4096 datagrams (drop newest)
path challenge validity	5 s
relay frame header / app payload	65 / 959 bytes
mesh hop TTL: default / max	3 / 6

Item	Value
Discovery and DHT	
discovery pass (desktop / Android)	30 s / 300 s
probes per pass / concurrency / timeout / budget	64 / 16 / 3 s / 15 s
DHT topic budgets (bootstrap, relays)	2 s, 3 s of 5 s
back-off (normal / preferred)	30 s→10 min / 5 s→60 s
announce interval (desktop / Android / daemon)	30 / 300 / 60 s
identity republish	300 s
DHT get timeout	5 s
index manifest / karma page size cap	950 / 950 bytes
received karma: stranger weight / min. age / half-life	0.1 / 7 epochs / 30 epochs
LAN beacon interval / expiry / max entries	10 s / 120 s / 128
host-cache eviction / write throttle	5 failures / 5 min
identity proof of work	20 leading zero bits of SHA-1(pub)
Mesh and services	
peer registry sync / digest refresh	30 s / 120 s
peer directory size / contact TTL / max failures	1024 / 10 min / 5
search: keywords / lim / path / TTL max	1024 B / 100 / 8 / 8
search: rows per chunk / chunks / bytes per holder	5 / 128 / 256 KiB
search: limits (ingress / per requester)	100 burst, 100/s / 5 burst, 5/s
replay cache	4096 entries, 601 s
request clock skew	±300 s
handler inbound frame	24,576 bytes
catalog: rows / pages / page bytes	100 / 64 / 937
catalog + metadata limit per requester	30 burst, 0.5/s
metadata: chunk / max size / chunks	448 B / 256 KiB / 586
metadata work budget / cache	1 MiB burst, 256 KiB/s / 32 for 5 min
outbound bytes (global)	2 MiB burst, 1 MiB/s
search timeout (desktop)	10 s
Control API	
body / poll batch / lookup	64 KiB / 256 msgs, 256 KiB / 100
queues: per consumer / consumers / total	512 / 256 / 4096 msgs, 16 MiB
`/register` clock skew	±60 s
desktop poll / supervision	300 ms / 10 s
Karma	
blocks per epoch / energy per epoch / decay / max	144 / 5 / 0.5 / 10
commit and publish interval	1800 s
remote manifest cap	64 KiB, 256 entries
tip cache / HTTP timeout	60 s / 5 s

A.3 Test vectors

These values were computed or checked independently of the Java code (SHA-1 with `shasum`; the identity signature with OpenSSL 3 Ed25519 verification over the re-encoded record without `sig`). The fuzz seed files under `common/fuzz/src/test/resources/com/frostwire/search/relay/` hold more signed samples: identity records v1–v3, karma entries and manifests (`KarmaChainEntryFuzzTestInputs`, `KarmaChainFuzzTestInputs`) and catalog pages v1 and v2 (`CatalogBrowserFuzzTestInputs`).

BEP 5 topic info-hashes (SHA-1 of the US-ASCII topic string):

```
frostwire-peers-v1      5e7d9baf693a90e2b02d126b23fdbf23926cf88f
frostwire-relays-v1     28f76105801c0cb8dcccf5b2e83badc9d65fc277
frostwire-bootstrap-v1  bf9f9c1d6ba04db30860d0f6e63df55e65aaa6fd
```

rUDP HELLO header (fuzz seed `seed-hello`; payload omitted):

```
46 77 02 01 11 22 33 44 55 66 77 88 00 00 00 00 00 00 00 00 a8
magic=Fw  version=2  type=HELLO  cid=0x1122334455667788  seq=0  ack=0  len=168
```

Signed identity record, version 4 (fuzz seed `IdentityRecordFuzzTestInputs/fromBencode/v4`). Concatenate the lines; the breaks are not part of the data. The public key is

```
b3c0b48c52553c67db5d54fec8eb0f696e60110bf61780c1d4b4113e672f4ec1
```

The signature verifies over the same dictionary without the `sig` entry. The `node_id` is *not* SHA-1 of the key, and the key has no proof of work; Java accepts the record anyway, which shows that neither is checked.

```
d4:capsi7e11:ed25519_pub43:s8C0jFJVPGfbXVT+yOsPaW5gEQv2F4DB1LQRPmcvTsE10
:first_seeni1767225600e6:ib_ver5:1.1.09:last_seeni1767229200e7:node_id40
:7a2767bbb9c5619afba7bc34d30643feaba539cb4:role4:BOTH9:rudp_porti6882e3:
sig86:pY1l3CkmLExgURqlY0kIZ0KUpiuJowY8YRhWuFWZMzqPw6E6g/RRKbdcHDTtysyC8D
T/E19PQo8oL8Jc80DCAA8:utp_porti6881e1:vi4e10:x25519_pub43:jUhhu1kRQd/x6i
Ngpo9Xy7GoOF1kUF1PwOMC8qH7Eige
```

`/register` signing string (`RegisterRequestVersionTest`): for pub text `abcd`, host `example.com`, port 6889, role `FORWARDER`, timestamp 1000 and version `1.1.0`, the signed UTF-8 string is `4:abcd|11:example.com|6889|FORWARDER|1000|5:1.1.0`.

LAN beacon. Accepted: `FWIB1 6888`, `FWIB1 65535`. Rejected: `FWIB1 70000`, `FWIB1 0`, `FWIB1 12a`, `FWIB1 -5`, `fwib1 6888`, `FWIB2 6888`, `FWIB1 6888 extra`, and anything over 64 bytes.

A.4 Discrepancies between documents and code

The code is authoritative. Earlier documents and comments say otherwise in these places:

- The 7.1.0 changelog lists `HOLE_PUNCH` for NAT traversal and a 5 s retransmission timeout. In v2, hole punching is disabled and the timeout is 15 s.
- `DESIGN_RELAY_REGISTRY.md` describes relays forwarding "encrypted blobs". No encryption exists. `ICEBRIDGE_REVISION_PLAN.md` correctly lists end-to-end encryption as not done.
- The code and documents call the DHT items "BEP 46". They are BEP 44 salted mutable items.
- `IncomingSearchRequestHandler` comments describe the public catalog as opt-in and off by default. The setting defaults to on.
- Comments describe search payloads as Base64url and bencode. They are standard Base64 and JSON.
- The topology compiled defaults ($M = 8$, search TTL 2) are replaced at run time by the role profile ($M = 32$ or 6, search TTL 7).
- `--rudp-port 0` binds an automatic port; the help text says it disables rUDP.

- The `SPEC_KARMA_CHAIN` document says entry timestamps are signed, lists a third block-hash source, and reverses hash bytes. The code does none of these.
- `ICEBRIDGE.md` and `desktop/ICEBRIDGE_REVISION_PLAN.md` say multi-hop search forwarding is disabled (`ttl=0`). It is enabled (`IncomingSearchRequestHandler.MULTI_HOP_FORWARDING_ENABLED = true`); forwarders keep the requester's signature and append themselves to the unsigned path.
- `DESIGN_RELAY_REGISTRY.md` still describes a TCP identity port (6888). It is gone: peers are verified over rUDP, and the relay port opens no socket.
- `ICEBRIDGE.md` says discovery probes 8 candidates at a time; the code uses 16 (`PeerDiscovery.DEFAULT_PROBE_CONCURRENCY`).
- The 7.1.0 changelog describes `/route` as "localhost-trusted unsigned"; it is token-protected and unsigned.